1 Vorspann: Sprachen

Vorlesung: 21.04.26

Def. 1.1: Ein *Alphabet* ist eine nicht leere, endliche Menge von *Zeichen*. ◇

Zeichen sind hier beliebige abstrakte Symbole.

Bsp.: für Alphabete, die in dieser Vorlesung, im täglichem Umgang mit Computern oder in der Forschung an unserem Lehrstuhl eine Rolle spielen

- $\{a, \ldots, z\}$
- $\{0, 1\}$
- $\{\mathit{rot}, \mathit{gelb}, \mathit{grün}\}$ (Ampelfarben)
- Die Menge aller ASCII-Symbole
- Die Menge aller Statements eines Computerprogramms

Wir verwenden typischerweise den griechischen Buchstaben Σ als Namen für ein Alphabet und die lateinischen Buchstaben a, b, c als Namen für Zeichen.Im Folgenden sei Σ immer ein beliebiges Alphabet.

Def. 1.2: Ein *Wort* ist eine endliche Folge von Elementen aus Σ. Die leere Folge nennen wir das *leere Wort* geschrieben als ε. Wir bezeichnen die Menge aller Wörter mit Σ^* und die Menge aller nicht leeren Wörter mit $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$.

Hier die induktive Definition.

1. $\varepsilon \in \Sigma^*$.
2. $a \in \Sigma$ und $w \in \Sigma^*$ impliziert $a.w \in \Sigma^*$.

◇

Hier einige Konventionen:

- Wir schreiben ein Wort immer ohne Trennsymbole wie z.B. Komma. Also `einhorn` statt `e,i,n,h,o,r,n`.
- Wir verwenden ε für das leere Wort.
- Wir verwenden typischerweise u, v, w als Namen für Wörter.

Bsp.: für Wörter über $\Sigma = \{a, \ldots, z\}$

- `rambo` (Länge 5)
- `eis`, `ies` (beide Länge 3 aber ungleich)
- ε (Länge 0)

Aus der induktiven Definition ergibt sich das folgende Beweisprinzip für Aussagen $\varphi(w)$ über Worte w:

$$\frac{\varphi(\varepsilon) \quad \forall a, w.\varphi(w) \Rightarrow \varphi(a.w)}{\forall w \in \Sigma^*.\varphi(w)}$$

Dieses Prinzip können wir auch verwenden um Funktionen auf Worten zu definieren. Als erstes Beispiel definieren wir die Länge eines Wortes, d.h. die Anzahl der Symbole des Wortes.

Def. 1.3 (Länge eines Worts)**:** Die *Länge* eines Worts, $|_| : \Sigma^* \to \mathbb{N}$, wird für alle $w \in \Sigma^*$ induktiv definiert durch

1. $|\varepsilon| = 0$;
2. für alle $a \in \Sigma$ und $w \in \Sigma^*$: $|a.w| = 1 + |w|$.

$\diamond$

Wörter lassen sich „verketten"/„aneinanderhängen". Die entsprechende Operation heißt *Konkatenation*, geschrieben „$\cdot$" (wie Multiplikation).

Def. 1.4 (Konkatenation von Wörtern)**:** Die *Konkatenation*, $\cdot : \Sigma^* \times \Sigma^* \to \Sigma^*$, wird für alle $v \in \Sigma^*$ induktiv definiert durch

1. $\varepsilon \cdot v = v$;
2. für alle $a \in \Sigma$ und $u \in \Sigma^*$: $(a.u) \cdot v = a.(u \cdot v)$.

$\diamond$

Lemma 1.1: Eigenschaften der Konkatenation: $(\Sigma^*, \varepsilon, \cdot)$ ist ein Monoid:

1. $\varepsilon \cdot v = v$ (leeres Wort ist linksneutrales Element);
2. $v \cdot \varepsilon = v$ (leeres Wort ist rechtsneutrales Element);
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ (Assoziativität).

Beweis: 1. $\varepsilon \cdot v = v$ (Definition)

2. $v \cdot \varepsilon = v$ Induktion über v:
   - IA: $(v = \varepsilon)$ $\varepsilon \cdot \varepsilon = \varepsilon$ (Definition)
   - IS: $(v \rightsquigarrow a.v)$ $(a.v) \cdot \varepsilon = a.(v \cdot \varepsilon) \overset{IV}{=} a.v$
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ Induktion über u:
   - IA: $(u = \varepsilon)$ $\varepsilon \cdot (v \cdot w) = (v \cdot w) = (\varepsilon \cdot v) \cdot w$ (Definition)
   - IS: $(u \rightsquigarrow a.u)$ $(a.u) \cdot (v \cdot w) \overset{Def}{=} a.(u \cdot (v \cdot w)) \overset{IV}{=} a.((u \cdot v) \cdot w) \overset{Def}{=} ((a.u) \cdot v) \cdot w$

$\square$

Bsp.:

- $\texttt{eis} \cdot \texttt{rambo} = \texttt{eisrambo}$
- $\texttt{rambo} \cdot \varepsilon = \texttt{rambo} = \varepsilon \cdot \texttt{rambo}$

Der Konkatenationsoperator „$\cdot$“ wird oft weggelassen (ähnlich wie der Multiplikationsoperator in der Arithmetik). Ebenso können durch die Assoziativität Klammern weggelassen werden.

$w_1 w_2 w_3$ steht also auch für $w_1 \cdot w_2 \cdot w_3$, für $(w_1 \cdot w_2) \cdot w_3$ und für $w_1 \cdot (w_2 \cdot w_3)$

Bemerkung: Die Zeichenfolge $\texttt{rambo}\varepsilon$ ist *kein* Wort. Diese Zeichenfolge ist lediglich eine Notation für eine Konkatenationsoperation, die ein Wort der Länge 5 (nämlich `rambo`) beschreibt.

Wörter lassen sich außerdem *potenzieren*:

Def. 1.5: Die *Potenzierung* von Wörtern, $\cdot^{\cdot} : \Sigma^* \times \mathbb{N} \to \Sigma^*$, ist induktiv definiert durch

1. $w^0 = \varepsilon$
2. $w^{n+1} = w \cdot w^n$ $\diamond$

Bsp.: $\texttt{eis}^3 \overset{(2.)}{=} \texttt{eis}\cdot\texttt{eis}^2 \overset{\text{zweimal (2.)}}{=} \texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}^0 \overset{(1.)}{=} \texttt{eis}\cdot\texttt{eis}\cdot\texttt{eis}\cdot\varepsilon = \texttt{eiseiseis}$

Def. 1.6: Eine *Sprache* über Σ ist eine Menge $L \subseteq \Sigma^*$. Gleichwertig: $L \in \mathcal{P}(\Sigma^*)$. $\diamond$

Bsp.:

- $\{\texttt{eis}, \texttt{rambo}\}$
- $\{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$
- $\{\}$ (die „leere Sprache“)
- $\{\varepsilon\}$ (ist verschieden von der leeren Sprache)
- Σ^*

Sämtliche Mengenoperationen sind auch Sprachoperationen, insbesondere Schnitt ($L_1 \cap L_2$), Vereinigung ($L_1 \cup L_2$), Differenz ($L_1 \setminus L_2$) und Komplement ($\overline{L_1} = \Sigma^* \setminus L_1$).

Weitere Operationen auf Sprachen sind Konkatenation und Potenzierung, sowie der *Kleene-Abschluss*.

Def. 1.7 (Konkatenation und Potenzierung von Sprachen)**:** Seien $U, V \subseteq \Sigma^*$. Dann ist die *Konkatenation* von U und V definiert durch

$$U \cdot V = \{uv \mid u \in U, v \in V\}$$

und die *Potenzierung* von U induktiv definiert durch

1. $U^0 = \{\varepsilon\}$
2. $U^{n+1} = U \cdot U^n$ ◇

Bsp.:

- $\{\texttt{eis}, \varepsilon\} \cdot \{\texttt{rambo}\} = \{\texttt{eisrambo}, \texttt{rambo}\}$
- $\{\texttt{eis}, \varepsilon\} \cdot \{\} = \{\}$
- $\{\}^0 = \{\varepsilon\}$
- $\{\}^4 = \{\}$
- $\{\texttt{eis}, \varepsilon\}^2 = \{\varepsilon, \texttt{eis}, \texttt{eiseis}\}$

Wie bei der Konkatenation von Wörtern dürfen wir den Konkatenationsoperator auch weglassen.

Def. 1.8 (Kleene-Abschluss, Kleene-Stern, Kleene-Plus)**:** Gegeben eine Sprache $U \subseteq \Sigma^*$, definieren wir den *Kleene-Abschluss* U^* wie folgt.

$$U^* = \bigcup_{n \in \mathbb{N}} U^n$$

Wie nennen den Operator $\cdot^* : \mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$, der für eine gegebene Sprache den Kleene-Abschluss liefert, den *Kleene-Stern*. Analog definieren wir den *Kleene-Plus* Operator $\cdot^+ : \mathcal{P}(\Sigma^*) \to \mathcal{P}(\Sigma^*)$ als

$$U^+ = \bigcup_{n \geq 1} U^n.$$

◇

Es gilt also $U^* = U^+ \cup \{\varepsilon\}$.

Zum Abschluss eine alternative Definition, die in manchen Textbüchern oder manchmal in der Literatur verwendet wird. In diesem Skript werden wir sie nicht weiter verwenden.

Def. 1.9 (Alternative Definition für Σ^*)**:** Sei $n \in \mathbb{N}$.

- $\Sigma^n = \{1, \ldots, n\} \to \Sigma$ Worte der Länge n
- insbesondere $\Sigma^0 = \{\} \to \Sigma$ enthält nur ein Element, ε
- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$
- Konkatenation: $u \in \Sigma^n$, $v \in \Sigma^m \Rightarrow u \cdot v \in \Sigma^{n+m}$ definiert durch

$$(u \cdot v)(i) = \begin{cases} u(i) & 1 \leq i \leq n \\ v(i-n) & n < i \leq n+m \end{cases}$$

◇

2 Reguläre Sprachen und endliche Automaten

Vorlesung: 22.04.26

Wie können wir potentiell unendlich große Mengen von Wörtern darstellen? Eine Lösung für dieses Problem sahen wir bereits im vorherigen Kapitel, als wir die (unendlich große) Menge der binär codierten Primzahlen mit Hilfe der folgenden Sprache darstellten.

$$L_{\mathsf{prim}} = \{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$$

Ein weiteres Beispiel ist die folgende Sprache.

$$L_{\mathsf{even}} = \{w \in \{0,1\}^* \mid \text{ die Anzahl der Einsen in } w \text{ ist gerade. }\}$$

Eine häufig interessante Fragestellung für ein gegebenes Wort w und eine Sprache L ist: „Ist w in L enthalten?“ (Also: „Gilt $w \in L$?“) Wir nennen dieses Entscheidungsproblem das *Wortproblem*. Eine konkrete Instanz des Wortproblems wäre z.B. „$1100101 \in L_{\mathsf{prim}}$?“ oder „$1100101 \in L_{\mathsf{even}}$?“

Die obige Darstellung der unendlichen Mengen L_{prim} und L_{even} ist zwar sehr kompakt, wir können daraus aber nicht direkt ein Vorgehen zur Lösung des Wortproblems ableiten. Wir müssen zunächst verstehen, was die Begriffe „Binärcodierung“, „Primzahl“ oder „gerade Anzahl“ bedeuten und für L_{prim} und L_{even} jeweils einen Algorithmus zur Entscheidung entwickeln.

In diesem Kapitel werden wir mit *endlichen Automaten* einen weiteren Formalismus kennenlernen, um (potentiell unendlich große) Mengen von Wörtern darzustellen. Ein Vorteil dieser Darstellung ist, dass es einen einheitlichen und effizienten Algorithmus für das Wortproblem gibt. Wir werden aber auch sehen, dass sich nicht jede Sprache (z.B. L_{prim}) mit Hilfe eines endlichen Automaten darstellen lässt.

2.1 Endliche Automaten

Wir beschreiben zunächst informell die Bestandteile eines endlichen Automaten:

Endliches Band (read-only; jede Zelle enthält ein $a_i \in \Sigma$; der Inhalt des Bands ist das *Eingabewort* bzw. die *Eingabe*)

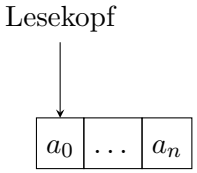


Abb. 1: Endliches Band

Lesekopf

- Der *Lesekopf* zeigt auf ein Feld des Bands, oder hinter das letzte Feld.
- Er bewegt sich feldweise nach rechts; andere Bewegungen (Vor- bzw. Zurückspulen) sind nicht möglich.
- Wenn er hinter das letzte Zeichen zeigt, *stoppt* der Automat. Er muss sich nun „entscheiden“ ob er das Wort *akzeptiert* oder nicht.

Zustände Zu jedem Zeitpunkt befindet sich der Automat in einem Zustand q aus einer *endlichen* Zustandsmenge Q.

Startzustand $q^{\mathsf{init}} \in Q$

Akzeptierende Zustände $F \subseteq Q$

Transitionsfunktion Im Zustand q beim Lesen von a gehe in Zustand $q' := \delta(q, a)$.

Der endliche Automat akzeptiert eine Eingabe, falls er in einem akzeptierenden Zustand stoppt.

Bsp. 2.1: Aufgabe:
„Erkenne alle Stapel von Macarons, in denen höchstens ein grünes Macaron vorkommt.“

[1]

Ein passendes Alphabet wäre $\Sigma = \{\texttt{grün}, \texttt{nicht-grün}\}$. Wir definieren die folgenden Zustände. (Die Metapher hier ist: „wenn ich mehr als ein grünes Macaron esse, wird mir übel, und das wäre nicht akzeptabel“.)

Zustand	Bedeutung
q_0	„alles gut“
q_1	„mir wird schon flau“
q_2	„mir ist übel“

Der Startzustand ist q_0. Akzeptierende Zustände sind q_0 und q_1. Die Transitionsfunktion δ ist durch die folgende Tabelle gegeben.

[1] Von links nach rechts:
By Mariajudit - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=48726001
By Michelle Naherny - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=44361114
By Keven Law - originally posted to Flickr as What's your Colour???, CC BY-SA 2.0, https://commons.wikimedia.org/w/index.php?curid=6851868

	grün	nicht-grün	
q_0	q_1	q_0	wechsle nach q_1 falls grün, ansonsten verweile
q_1	q_2	q_1	wechsle nach q_2 falls grün, ansonsten verweile
q_2	q_2	q_2	verweile, da es nichts mehr zu retten gibt

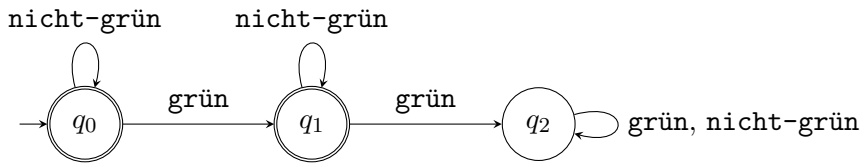
Def. 2.1 (DEA)**:** Ein *deterministischer endlicher Automat (DEA)*[2] ist ein 5-Tupel

$$\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \to Q$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\mathsf{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende* Zustände nennen. ◇

DEAs lassen sich auch graphisch darstellen. Dabei gibt man für den Automaten einen gerichteten Graphen an. Die Knoten des Graphen sind die Zustände, und mit Zeichen beschriftete Kanten zeigen, welchen Zustandsübergang die Transitionsfunktion für das nächste Zeichen erlaubt. Der Startzustand ist mit einem unbeschrifteten Pfeil markiert, akzeptierende Zustände sind doppelt eingekreist. Hier ist die graphische Darstellung von $A_{\texttt{Macaron}}$ aus Beispiel 2.1:



Die folgenden beiden Definitionen erlauben uns mit Hilfe eines DEA eine Sprache zu charakterisieren. Hierbei sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F)$ ein DEA.

Def. 2.2: Die *induktive Erweiterung* von $\delta : Q \times \Sigma \to Q$ auf Wörter, $\tilde{\delta} : Q \times \Sigma^* \to Q$, ist (induktiv) definiert durch

1. $\tilde{\delta}(q, \varepsilon) = q$ (Wortende erreicht)

[2] engl. DFA $\hat{=}$ deterministic finite automaton

2. $\tilde{\delta}(q, aw) = \tilde{\delta}(\delta(q, a), w)$ (Rest im Folgezustand verarbeiten) ◇

Def. 2.3: Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F)$. Ein Wort $w \in \Sigma^*$ wird von $\mathcal{A}$ *akzeptiert*, falls $\tilde{\delta}(q^{\mathsf{init}}, w) \in F$. Die *von $\mathcal{A}$ akzeptierte Sprache*, geschrieben $L(\mathcal{A})$, ist die Menge aller Wörter, die von $\mathcal{A}$ akzeptiert werden. D.h.,

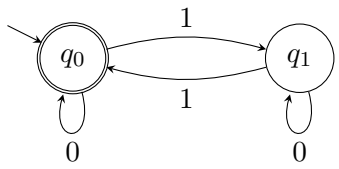
$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \tilde{\delta}(q^{\mathsf{init}}, w) \in F\}.$$

Eine durch einen DEA akzeptierte Sprache heißt *regulär*. ◇

Bsp. 2.2: Ein möglicher DEA für die Sprache

$$L_{\mathsf{even}} = \{w \in \{0,1\}^* \mid \text{ Die Anzahl der Einsen in } w \text{ ist gerade. }\}$$

aus der Einleitung dieses Kapitels hat die folgende graphische Repräsentation.



Frage: Angenommen wir haben zwei DEAs A_1 und A_2, können wir dann immer einen DEA konstruieren der den Schnitt $L(A_1) \cap L(A_2)$ akzeptiert?

Idee: Lasse beide DEAs parallel laufen, akzeptiere nur wenn beide DEAs akzeptieren.

Frage: Lassen sich zwei *parallel laufende* DEAs in einem einzigen DEA implementieren?

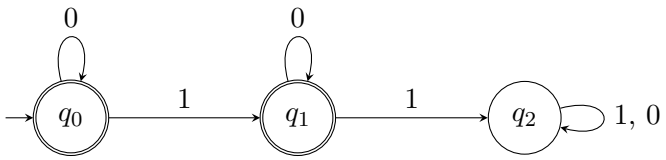
Die nächste Konstruktion und der drauf folgende Satz liefern eine positive Antwort auf diese Frage.

Def. 2.4 (Produktautomat für Schnitt)**:** Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\mathsf{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\mathsf{init}}, F_2)$ DEAs. Wir definieren den *Produktautomaten für Schnitt* als den DEA $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\mathsf{init}}, F_\cap)$ mit den folgenden Komponenten.

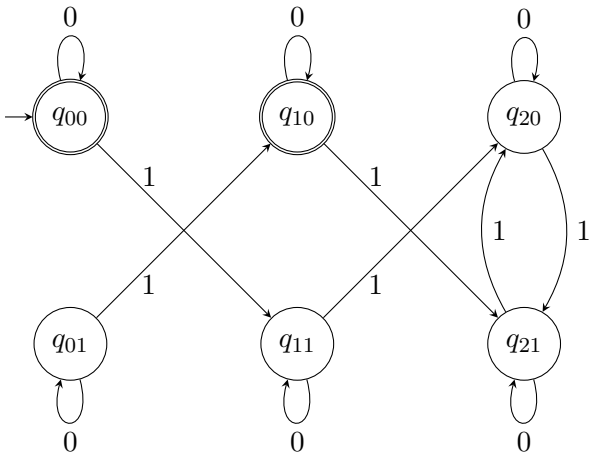
$$\begin{aligned} Q_\cap &= Q_1 \times Q_2 \\ \delta_\cap((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, a)) \ , \quad \text{für alle } a \in \Sigma \\ q_\cap^{\mathsf{init}} &= (q_1^{\mathsf{init}}, q_2^{\mathsf{init}}) \\ F_\cap &= F_1 \times F_2 \end{aligned}$$ ◇

Vorlesung: 28.04.26

Im folgenden Beispiel sei A_1 der DEA über dem Alphabet $\Sigma = \{0, 1\}$, dessen graphische Repräsentation nahezu mit $A_{\texttt{Macaron}}$ identisch ist.



Bsp. 2.3: Der Produktautomat für Schnitt von A_1 und A_{even} hat die folgende graphische Repräsentation, wobei wir um Platz zu sparen „q_{ij}“ statt „(q_i, q_j)“ schreiben.



Satz 2.1: Für beliebige DEAs A_1 und A_2 akzeptiert der entsprechende Produktautomat für Schnitt die Sprache $L(A_1) \cap L(A_1)$.

Beweis: [3] Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\mathsf{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\mathsf{init}}, F_2)$ DEAs und $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\mathsf{init}}, F_\cap)$ der zugehörige Produktautomat für Schnitt. Wir zeigen zunächst per Induktion über w, dass für alle $w \in \Sigma^*$, für alle $q_1 \in Q_1$ und für alle $q_2 \in Q_2$ die folgende Gleichung $\varphi(w)$ gilt.

$$\varphi(w) := \tilde{\delta}_\cap((q_1, q_2), w) = (\tilde{\delta}_1(q_1, w), \tilde{\delta}_2(q_2, w))$$

Der Induktionsanfang $\varphi(\varepsilon)$ folgt dabei direkt aus Def. 2.2.

$$\tilde{\delta}_\cap((q_1, q_2), \varepsilon) = (q_1, q_2)$$

[3] Dieser erste Beweis ist außergewöhnlich detailliert. In den folgenden Beweisen werden wir einfache Umformungen zusammenfassen.

Den Induktionsschritt $\varphi(w) \rightsquigarrow \varphi(a.w)$ zeigen wir mit Hilfe der folgenden Umformungen, wobei $a \in \Sigma$ ein beliebiges Zeichen und $w \in \Sigma^*$ ein beliebiges Wort.

$$\begin{aligned}
\tilde{\delta}_\cap((q_1, q_2), aw) &\overset{\text{Def. 2.2}}{=} \tilde{\delta}_\cap(\delta_\cap((q_1, q_2), a), w) \\
&\overset{\text{Def. } \delta_\cap}{=} \tilde{\delta}_\cap((\delta_1(q_1, a), \delta_2(q_2, a)), w) \\
&\overset{\text{I.V.}}{=} (\tilde{\delta}_1(\delta_1(q_1, a), w), \tilde{\delta}_2(\delta_2(q_2, a), w)) \\
&\overset{\text{Def. 2.2}}{=} (\tilde{\delta}_1(q_1, aw), \tilde{\delta}_2(q_2, aw))
\end{aligned}$$

Schließlich zeigen wir $L(A_\cap) = L(A_1) \cap L(A_2)$ mit Hilfe der folgenden Umformungen für ein beliebiges $w \in \Sigma^*$.

$$\begin{aligned}
w \in L(A_\cap) &\overset{\text{Def. 2.3}}{\text{gdw}} \tilde{\delta}_\cap(q_\cap^{\mathsf{init}}, w) \in F_\cap \\
&\overset{\text{Def. } q_\cap^{\mathsf{init}}}{\text{gdw}} \tilde{\delta}_\cap((q_1^{\mathsf{init}}, q_2^{\mathsf{init}}), w) \in F_\cap \\
&\text{gdw} \quad (\tilde{\delta}_1(q_1^{\mathsf{init}}, w), \tilde{\delta}_2(q_2^{\mathsf{init}}, w)) \in F_\cap \\
&\overset{\text{Def. } F_\cap}{\text{gdw}} \tilde{\delta}_1(q_1^{\mathsf{init}}, w) \in F_1 \text{ und } \tilde{\delta}_2(q_2^{\mathsf{init}}, w) \in F_2 \\
&\overset{\text{Def. 2.3}}{\text{gdw}} w \in L(A_1) \text{ und } w \in L(A_2)
\end{aligned}$$

□

2.2 Minimierung endlicher Automaten

Beobachtung: Der Zustand q_{01} im Beispiel 2.3 scheint nutzlos. Wir charakterisieren die Nützlichkeit eines Zustands in einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\mathsf{init}}, F)$ formal wie folgt.

Def. 2.5: Ein Zustand $q \in Q$ heißt *erreichbar*, falls ein $w \in \Sigma^*$ existiert, sodass $\tilde{\delta}(q^{\mathsf{init}}, w) = q$. ◇

Bemerkung: Die Menge der erreichbaren Zustände kann mit dem folgenden Verfahren in $O(|Q| \cdot |\Sigma|)$ berechnet werden.

- Fasse $\mathcal{A}$ als Graph auf.
- Wende Tiefensuche ab q^{init} an und markiere dabei alle besuchten Zustände.
- Genau die markierten Zustände sind erreichbar.

Beobachtung: Auch nach dem Entfernen der nicht erreichbaren Zustände q_{01} und q_{10} scheint der DEA aus Bsp. 2.3 unnötig groß zu sein. Das Verhalten des DEA in den Zuständen q_{11}, q_{20} und q_{21} ist sehr ähnlich. Wir charakterisieren diese „Ähnlichkeit“ formal wie folgt.

Def. 2.6: Zwei Zustände $p, q \in Q$ eines DEA sind *äquivalent*, geschrieben $p \equiv q$, falls

$$\forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \text{ gdw } \tilde{\delta}(q, w) \in F \qquad \diamond$$

Bsp. 2.4: Für Bsp. 2.3 gilt: Die Zustände q_{11}, q_{20} und q_{21} sind paarweise äquivalent. Die Zustände q_{00} und q_{10} sind äquivalent. Keine weiteren Zustandspaare sind äquivalent.

Geschrieben als Menge von Paaren sieht die Relation $\equiv \subseteq Q \times Q$ also wie folgt aus:

$$\{\, (q_{00}, q_{10}), (q_{10}, q_{00}),\ (q_{11}, q_{20}), (q_{20}, q_{11}),\ (q_{20}, q_{21}), (q_{21}, q_{20}),\ (q_{21}, q_{11}), (q_{11}, q_{21}) \,\}$$

Lemma 2.2: Die Relation „$\equiv$“ ist eine *Äquivalenzrelation.*

Beweis: Zu zeigen ist, dass $\equiv$ reflexiv, transitiv und symmetrisch ist.

- Die Relation $\equiv$ ist offensichtlich reflexiv.
- Die Symmetrie und Transitivität von $\equiv$ folgen aus der Symmetrie und Transitivität der logischen Interpretation von „genau dann, wenn“ (gdw).

$\square$

Bsp. 2.5: Für den DEA aus Bsp. 2.3 hat die Relation $\equiv$ drei Äquivalenzklassen.[4]

$$\begin{aligned}
[q_{00}] &= \{q_{00}, q_{10}\}, \\
[q_{01}] &= \{q_{01}\}, \\
[q_{11}] &= \{q_{11}, q_{20}, q_{21}\}
\end{aligned}$$

Idee: „Verschmelze“ alle Zustände aus einer Äquivalenzklasse zu einem einzigen Zustand. Bedenken: Bei einem DEA hat jeder Zustand hat für jedes Zeichen einen Nachfolger. Wenn wir Zustände verschmelzen, könnte es mehrere Nachfolger geben und das Resultat wäre kein wohldefinierter DEA mehr.

Das folgende Lemma zeigt, dass unsere Bedenken nicht gerechtfertigt sind. Sind zwei Zustände äquivalent, so sind auch für jedes Zeichen ihre Nachfolger äquivalent.

Lemma 2.3: Für alle $p, q \in Q$ gilt:

$$p \equiv q \quad \Rightarrow \quad \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a)$$

[4] Den Zustand q_{11} als Repräsentanten für die dritte Äquivalenzklasse zu wählen ist eine völlig willkürliche Entscheidung. Wir könnten genauso gut q_{20} oder q_{21} wählen.

Beweis:

$$
\begin{array}{lll}
p \equiv q & \text{gdw} & \forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \Leftrightarrow \tilde{\delta}(q, w) \in F \\
 & \text{gdw} & (p \in F \Leftrightarrow q \in F) \wedge \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(p, aw) \in F \Leftrightarrow \tilde{\delta}(q, aw) \in F \\
 & \text{impliziert} & \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(\delta(p, a), w) \in F \Leftrightarrow \tilde{\delta}(\delta(q, a), w) \in F \\
 & \text{gdw} & \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a)
\end{array}
$$

□

Wir formalisieren das „Verschmelzen“ von Zuständen wie folgt.

Def. 2.7: Der Äquivalenzklassenautomat $\mathcal{A}_\equiv = (Q_\equiv, \Sigma, \delta_\equiv, q^{\mathsf{init}}{}_\equiv, F_\equiv)$ zu einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\mathsf{init}}, F)$ ist bestimmt durch:

$$
\begin{array}{ll}
Q_\equiv = \{[q] \mid q \in Q\} & \delta_\equiv([q], a) = [\delta(q, a)] \\
q^{\mathsf{init}}_\equiv = [q^{\mathsf{init}}] & F_\equiv = \{[q] \mid q \in F\}
\end{array}
$$

◇